

# OPERATING SYSTEMS LAB

## Assignment – 5

### Process Synchronization using xv6

#### Question 1 – Mutual Exclusion using Peterson's Algorithm

The assignment explicitly requires the shared flag[2] and turn variables and the waiting condition flag[other] == 1 && turn == other. No semaphore or OS lock is used for Peterson's critical-section decision.

#### Data structure and logic

```
struct shared_data {  
    int flag[2];  
    int turn;  
    int shared_counter;  
};
```

For process i, let other = 1 - i.

1. Set flag[i] = 1 to announce the intention to enter the critical section.
2. Set turn = other, giving the other process priority if both processes request entry.
3. Busy-wait while flag[other] == 1 && turn == other.
4. Enter the critical section and increment shared\_counter.
5. Set flag[i] = 0 when leaving the critical section.
6. Execute a short remainder-section delay before the next iteration.

#### Why Peterson's Algorithm works

The flag array records interest in the critical section. If only one process is interested, it enters immediately. If both are interested, turn determines which process waits. Because each process sets turn to the other process before waiting, both cannot simultaneously satisfy the conditions required to enter the critical section. Thus only one process updates shared\_counter at a time.

The counter is shared, so both processes update the same memory location. With 2 processes × 10 iterations, the expected final value is 20. A value below 20 would indicate a lost update.

## Q1 output screenshot

```
Process 0 in CS, counter = 1
Process 0 exiting CS
Process 1 in CS, counter = 2
Process 1 exiting CS
Process 0 in CS, counter = 3
Process 0 exiting CS
Process 1 in CS, counter = 4
Process 1 exiting CS
Process 1 in CS, counter = 5
Process 1 exiting CS
Process 0 in CS, counter = 6
Process 0 exiting CS
Process 0 in CS, counter = 7
Process 0 exiting CS
Process 1 in CS, counter = 8
Process 1 exiting CS
Process 0 in CS, counter = 9
Process 0 exiting CS
Process 1 in CS, counter = 10
Process 1 exiting CS
Process 1 in CS, counter = 11
Process 1 exiting CS
Process 0 in CS, counter = 12
Process 0 exiting CS
Process 0 in CS, counter = 13
Process 0 exiting CS
Process 1 in CS, counter = 14
Process 1 exiting CS
Process 0 in CS, counter = 15
Process 0 exiting CS
Process 0 in CS, counter = 16
Process 0 exiting CS
Process 1 in CS, counter = 17
Process 1 exiting CS
Process 0 in CS, counter = 18
Process 0 exiting CS
Process 0 in CS, counter = 19
Process 0 exiting CS
Process 0 in CS, counter = 20
Process 0 exiting CS
Final counter = 20
$
```

## Question 2 – Producer–Consumer Problem (Bounded Buffer)

### Synchronization variables

```
#define EMPTY 0
#define FULL 1
#define MUTEX 2
```

Conceptually, empty represents available buffer capacity, full represents available produced items, and mutex serializes the protected producer/consumer operation. The default bounded-buffer capacity is 5 as specified by the assignment.

### Custom lightweight semaphore

The semaphore implementation is kernel-backed and uses xv6's sleep/wakeup mechanism. The semaphore lock protects the semaphore count. If the count is zero, the process prepares to sleep, releases the semaphore lock, sleeps, and then retries. This avoids busy-waiting.

```
// kernel/semaphore.c
#include "types.h"
#include "riscv.h"
#include "spinlock.h"
#include "defs.h"

#define NSEM 3

struct semaphore {
    struct spinlock lock;
    int value;
};

struct semaphore semaphores[NSEM];

void
sem_init(int id, int value)
{
    if(id < 0 || id >= NSEM)
        return;

    initlock(&semaphores[id].lock, "semaphore");
    semaphores[id].value = value;
}

void
sem_wait(int id)
{
    if(id < 0 || id >= NSEM)
        return;

    acquire(&semaphores[id].lock);

    while(semaphores[id].value == 0){
        sleep_prepare(&semaphores[id]);
        release(&semaphores[id].lock);

        sleep();

        acquire(&semaphores[id].lock);
    }

    semaphores[id].value--;
    release(&semaphores[id].lock);
}
```

```

}

void
sem_post(int id)
{
    if(id < 0 || id >= NSEM)
        return;

    acquire(&semaphores[id].lock);

    semaphores[id].value++;
    wakeup(&semaphores[id]);

    release(&semaphores[id].lock);
}

```

## Semaphore system calls

```

// kernel/defs.h
void sem_init(int id, int value);
void sem_wait(int id);
void sem_post(int id);

// kernel/syscall.h
#define SYS_sync      22
#define SYS_sem_init  23
#define SYS_sem_wait  24
#define SYS_sem_post  25

```

The exact syscall numbers can differ between branches because Q1 and Q2 were developed independently. The important design is that user-space `sem_init/sem_wait/sem_post` wrappers call kernel semaphore implementations backed by `sleep_prepare()`, `sleep()`, and `wakeup()`.

```

// kernel/sysproc.c
uint64
sys_sem_init(void)
{
    int id, value;
    argint(0, &id);
    argint(1, &value);
    sem_init(id, value);
    return 0;
}

uint64
sys_sem_wait(void)
{
    int id;
    argint(0, &id);
    sem_wait(id);
    return 0;
}

uint64
sys_sem_post(void)
{
    int id;
    argint(0, &id);
    sem_post(id);
    return 0;
}

```

## Q2 output screenshot

```
laasya@laasyaSR1:~/OS_LABS/xv6-F1scv$ make qemu
```

```
Producer: produced 1
Consumer: consumed 1
Producer: produced 2
Producer: produced 3
Consumer: consumed 2
Producer: produced 4
Producer: produced 5
Consumer: consumed 3
Producer: produced 6
Producer: produced 7
Consumer: consumed 4
Producer: produced 8
Consumer: consumed 5
Producer: produced 9
Producer: produced 10
Consumer: consumed 6
Producer: produced 11
Consumer: consumed 7
Producer: produced 12
Consumer: consumed 8
Producer: produced 13
Consumer: consumed 9
Producer: produced 14
Consumer: consumed 10
Producer: produced 15
Consumer: consumed 11
Producer: produced 16
Consumer: consumed 12
Producer: produced 17
```

```
Producer: produced 17
Consumer: consumed 13
Producer: produced 18
Consumer: consumed 14
Producer: produced 19
Consumer: consumed 15
Producer: produced 20
Consumer: consumed 16
Consumer: consumed 17
Consumer: consumed 18
Consumer: consumed 19
Consumer: consumed 20
```

```
$
```

## Question 3 – Readers–Writers Problem

### Shared state

```
struct shared_state {
    int shared_data;
    int read_count;
};

volatile struct shared_state *shared;
```

### Semaphore roles

```
#define RMUTEX      0
#define RESOURCE    1
#define SERVICE_QUEUE 2
#define PRINT_MUTEX 3
```

- RMUTEX: protects increments/decrements of read\_count.
- RESOURCE: provides exclusive access to shared\_data for writers and to the first/last-reader boundary.
- SERVICE\_QUEUE: fair turnstile; prevents a stream of new readers from bypassing waiting writers forever.
- PRINT\_MUTEX: prevents concurrent printf calls from visually interleaving in the xv6 terminal.

### Reader algorithm

```
wait(SERVICE_QUEUE);
wait(RMUTEX);

read_count++;

if(read_count == 1)
    wait(RESOURCE);

signal(SERVICE_QUEUE);
signal(RMUTEX);

/* READ shared_data */

wait(RMUTEX);

read_count--;

if(read_count == 0)
    signal(RESOURCE);

signal(RMUTEX);
```

The first reader locks RESOURCE, thereby preventing writers. Additional readers only increment read\_count, so they can enter concurrently. The last reader releases RESOURCE, allowing a writer to proceed.

### Writer algorithm

```
wait(SERVICE_QUEUE);
wait(RESOURCE);
signal(SERVICE_QUEUE);

/* WRITE shared_data */

signal(RESOURCE);
```

A writer obtains RESOURCE before modifying shared\_data. Therefore no other writer and no active reader can access the shared resource during the write.

### Why basic readers-priority can starve writers

In the classic readers-priority solution using only rmutex and resource, a new reader may enter as long as readers are already active. If readers keep arriving before the current reader group becomes empty, the resource may remain occupied indefinitely and a waiting writer can starve.

The serviceQueue turnstile solves this problem by forcing readers and writers to pass through a common admission point. Once a writer is waiting at the queue, newly arriving readers cannot continuously overtake it. Already admitted readers may continue together, preserving concurrent reader execution, while the waiting writer eventually receives RESOURCE.

### Build/Makefile integration

```
# Makefile
```

```
OBJS += $K/semaphore.o
```

```
# User program list
$U/_readwrite\
```

The Makefile includes the kernel semaphore object and the readwrite user program so that both the kernel support and user executable are included in the xv6 build/filesystem image.

## Q3 Output

```
hart 1 starting
hart 2 starting
init: starting sh
$ readwrite
[38] Reader 1 (PID 4) waiting
[38] Reader 1 (PID 4) START: data=0, readers=1
[38] Writer 2 (PID 8) waiting
[38] Reader 2 (PID 5) waiting
[39] Reader 3 (PID 6) waiting
[39] Writer 1 (PID 7) waiting
[40] Reader 1 (PID 4) END
[40] Writer 2 (PID 8) START: old data=0
[41] Reader 1 (PID 4) waiting
[42] Writer 2 (PID 8) END: new data=1
[42] Reader 2 (PID 5) START: data=1, readers=1
[42] Reader 1 (PID 4) START: data=1, readers=3
[42] Reader 3 (PID 6) START: data=1, readers=3
[43] Writer 2 (PID 8) waiting
[44] Reader 1 (PID 4) END
[44] Reader 2 (PID 5) END
[45] Reader 1 (PID 4) waiting
[45] Reader 2 (PID 5) waiting
[45] Reader 3 (PID 6) END
[45] Writer 1 (PID 7) START: old data=1
[46] Reader 3 (PID 6) waiting
[47] Writer 1 (PID 7) END: new data=2
[47] Reader 1 (PID 4) START: data=2, readers=1
[47] Reader 2 (PID 5) START: data=2, readers=3
[47] Reader 3 (PID 6) START: data=2, readers=3
[48] Writer 1 (PID 7) waiting
[49] Reader 3 (PID 6) END
[49] Reader 1 (PID 4) END
[49] Reader 2 (PID 5) END
[49] Writer 2 (PID 8) START: old data=2
[50] Reader 1 (PID 4) waiting
[49] Reader 2 (PID 5) END
[49] Writer 2 (PID 8) START: old data=2
[50] Reader 1 (PID 4) waiting
[50] Reader 2 (PID 5) waiting
[50] Reader 3 (PID 6) waiting
[51] Writer 2 (PID 8) END: new data=3
[51] Writer 1 (PID 7) START: old data=3
[52] Writer 2 (PID 8) waiting
[53] Writer 1 (PID 7) END: new data=4
[53] Reader 1 (PID 4) START: data=4, readers=1
[53] Reader 2 (PID 5) START: data=4, readers=3
[53] Reader 3 (PID 6) START: data=4, readers=3
[54] Writer 1 (PID 7) waiting
[55] Reader 1 (PID 4) END
[55] Reader 2 (PID 5) END
[56] Reader 1 (PID 4) waiting
[56] Reader 2 (PID 5) waiting
[56] Reader 3 (PID 6) END
[56] Writer 2 (PID 8) START: old data=4
[57] Reader 3 (PID 6) waiting
[58] Writer 2 (PID 8) END: new data=5
[58] Reader 1 (PID 4) START: data=5, readers=1
[58] Reader 2 (PID 5) START: data=5, readers=3
[58] Reader 3 (PID 6) START: data=5, readers=3
[59] Writer 2 (PID 8) waiting
[60] Reader 1 (PID 4) END
[60] Reader 2 (PID 5) END
[60] Reader 3 (PID 6) END
[60] Writer 1 (PID 7) START: old data=5
[61] Reader 2 (PID 5) waiting
[61] Reader 3 (PID 6) waiting
[62] Writer 1 (PID 7) END: new data=6
[62] Writer 2 (PID 8) START: old data=6
[63] Writer 1 (PID 7) waiting
[64] Writer 2 (PID 8) END: new data=7
[63] Writer 1 (PID 7) waiting
[64] Writer 2 (PID 8) waiting
[64] Writer 2 (PID 8) END: new data=7
[64] Reader 2 (PID 5) START: data=7, readers=1
[64] Reader 3 (PID 6) START: data=7, readers=2
[65] Writer 2 (PID 8) waiting
[66] Reader 2 (PID 5) END
[66] Reader 3 (PID 6) END
[66] Writer 1 (PID 7) START: old data=7
[68] Writer 1 (PID 7) END: new data=8
[68] Writer 2 (PID 8) START: old data=8
[69] Writer 1 (PID 7) waiting
[70] Writer 2 (PID 8) END: new data=9
[70] Writer 1 (PID 7) START: old data=9
[72] Writer 1 (PID 7) END: new data=10
Final shared_data = 10
Final read_count = 0
$ []
```



## Q4- Dining Philosophers Problem (Deadlock Avoidance)

The Dining Philosophers problem was implemented using **five binary semaphores**, one for each fork, and a `ROOM` semaphore initialized to 4. Each philosopher repeatedly goes through the states **Thinking** → **Hungry** → **Eating**. A philosopher must acquire the `ROOM` semaphore and then both its left and right forks before eating. After eating, both forks and the room slot are released. Allowing at most four philosophers to compete for forks at a time prevents the circular-wait condition and hence avoids deadlock. The output shows all five philosophers successfully completing five eating cycles, with neighboring philosophers never eating simultaneously and non-neighboring philosophers being able to eat concurrently.

```
Philosopher 0: THINKING
Philosopher 1: THINKING
Philosopher 2: THINKING
Philosopher 3: THINKING
Philosopher 4: THINKING
Philosopher 2: HUNGRY
Philosopher 2: EATING
Philosopher 3: HUNGRY
Philosopher 0: HUNGRY
Philosopher 0: EATING
Philosopher 1: HUNGRY
Philosopher 4: HUNGRY
Philosopher 2: THINKING
Philosopher 3: EATING
Philosopher 0: THINKING
Philosopher 1: EATING
Philosopher 0: HUNGRY
Philosopher 1: THINKING
Philosopher 2: HUNGRY
Philosopher 2: EATING
Philosopher 3: THINKING
Philosopher 4: EATING
Philosopher 1: HUNGRY
Philosopher 2: THINKING
Philosopher 3: HUNGRY
Philosopher 2: EATING
Philosopher 3: THINKING
Philosopher 4: THINKING
Philosopher 1: THINKING
Philosopher 0: HUNGRY
Philosopher 1: EATING
Philosopher 2: HUNGRY
Philosopher 2: EATING
Philosopher 3: THINKING
Philosopher 0: THINKING
Philosopher 4: HUNGRY
Philosopher 4: EATING
Philosopher 2: THINKING
Philosopher 2: HUNGRY
Philosopher 3: HUNGRY
Philosopher 0: HUNGRY
Philosopher 0: EATING
Philosopher 1: HUNGRY
Philosopher 0: HUNGRY
Philosopher 1: EATING
Philosopher 2: THINKING
Philosopher 3: EATING
Philosopher 4: THINKING
Philosopher 1: THINKING
Philosopher 0: EATING
Philosopher 3: EATING
Philosopher 4: THINKING
Philosopher 1: THINKING
Philosopher 0: EATING
Philosopher 3 completed all 5 cycles
Philosopher 4: HUNGRY
Philosopher 1: HUNGRY
Philosopher 1: EATING
Philosopher 4: EATING
Philosopher 0 completed all 5 cycles
Philosopher 1 completed all 5 cycles
Philosopher 4 completed all 5 cycles
All philosophers completed successfully.
```

## Conclusion

This assignment provided practical experience in implementing and understanding process synchronization mechanisms in the xv6 operating system. **Peterson's Algorithm** demonstrated how mutual exclusion can be achieved between two processes using only shared variables such as `flag[]` and `turn`. The **Producer-Consumer problem** demonstrated the use of counting semaphores and mutex synchronization to coordinate access between a producer and a consumer while preventing data loss and incorrect ordering. The **Readers-Writers problem** demonstrated how multiple readers can access shared data concurrently while writers are given exclusive access, with a fair synchronization mechanism used to prevent indefinite writer starvation.

Overall, the implementation helped demonstrate the importance of synchronization in concurrent systems and how mechanisms such as **shared memory, semaphores, mutual exclusion, and sleep/wakeup** can be integrated into an operating system. The successful execution and verification of all three programs in xv6 confirm that the implemented synchronization mechanisms provide the required coordination and protection for shared resources.